

Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Content

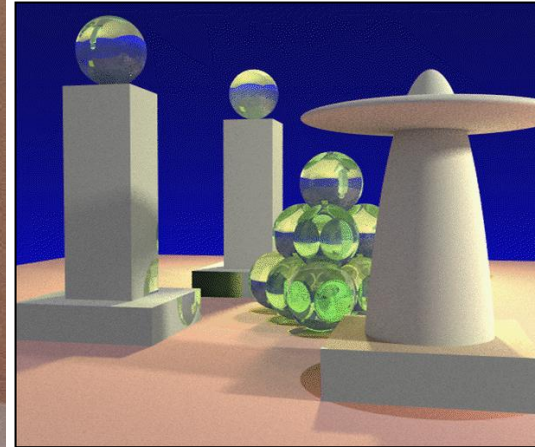
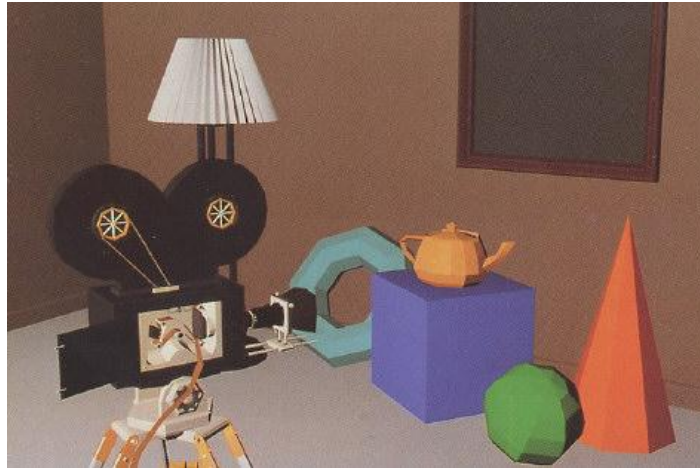
- ▶ Graphics Areas
- ▶ Major Applications
- ▶ Graphics APIs
- ▶ Graphics Pipeline
- ▶ Numerical Issues
- ▶ Efficiency
- ▶ Designing and Coding Graphics Programs

Computer graphics

- ▶ The term *computer graphics* describes any use of computers to create and manipulate images can be used to create all kinds of images—realistic visual effects, informative technical illustrations, or beautiful computer animations.
- ▶ Graphics can be two- or three-dimensional;
- ▶ images can be completely synthetic or can be produced by manipulating photographs.

History

- ▶ Brief history of significant developments in field
- ▶ End with a video showcasing graphics



The term Computer Graphics was coined by William Fetter of Boeing in 1960
First graphic system in mid 1950s USAF SAGE radar data (developed MIT)

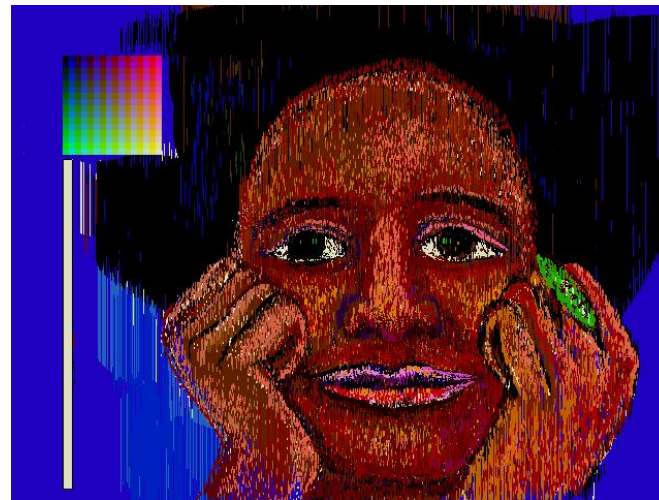
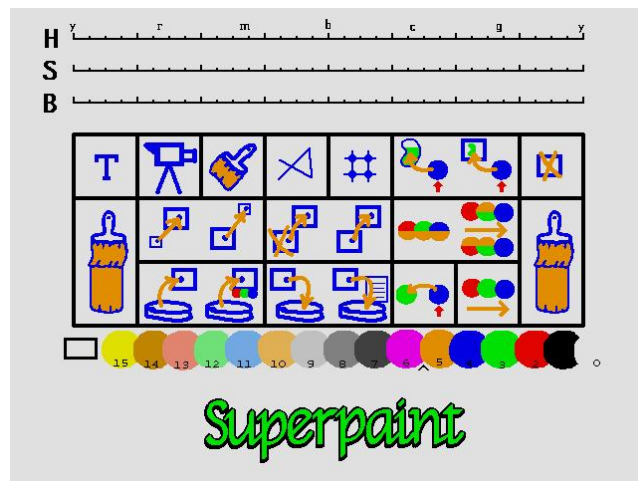
Drawing: Sketchpad (1963)

- ▶ Sketchpad (Sutherland, MIT 1963)
- ▶ First interactive graphics system
- ▶ Many of concepts for drawing in current systems
 - ▶ Pop up menus
 - ▶ Constraint-based drawing
 - ▶ Hierarchical Modeling



Paint Systems

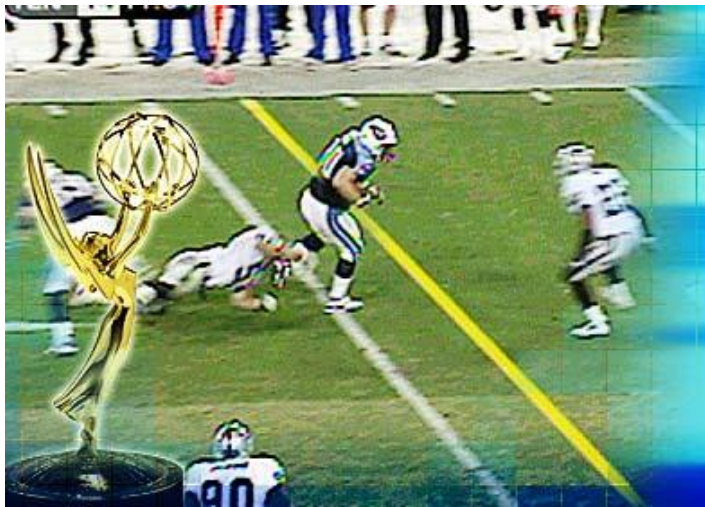
- ▶ SuperPaint system: Richard Shoup, Alvy Ray Smith (PARC, 1973-79)



- ▶ Nowadays, image processing programs like Photoshop can draw, paint, edit, etc.

Image Processing

- ▶ Digitally alter images, crop, scale, composite
- ▶ Add or remove objects
- ▶ Sports broadcasts for TV (combine 2D and 3D processing)



Graphics Areas

- ▶ Most graphics practitioners would agree on the following major areas of computer graphics
- ▶ **Modeling** deals with the mathematical specification of shape and appearance properties in a way that can be stored on the computer. For example a coffee mug might be described as a set of ordered 3D points along with some interpolation rule to connect the points and a reflection model that describes how light interacts with the mug.
- ▶ **Rendering** is a term inherited from art and deals with the creation of shaded images from 3D computer models.
- ▶ **Animation** is a technique to create an illusion of motion through sequences of images. Animation uses modeling and rendering but adds the key issue of movement over time, which is not usually dealt with in basic modeling and rendering.

3D Graphics Pipeline

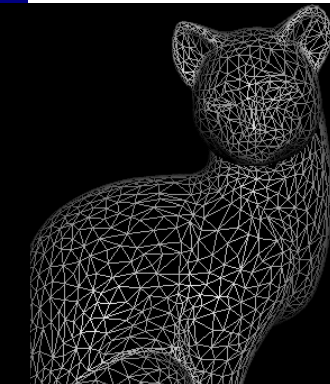
Modeling



Animation

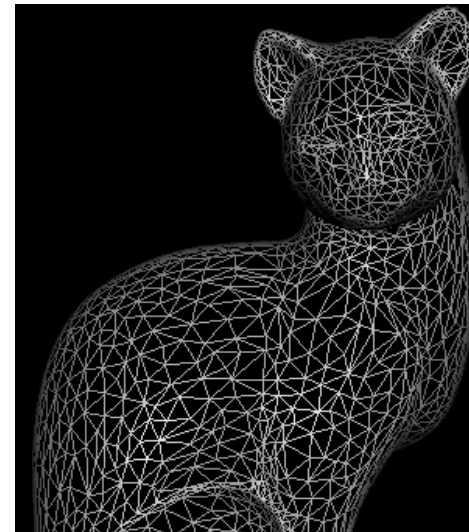
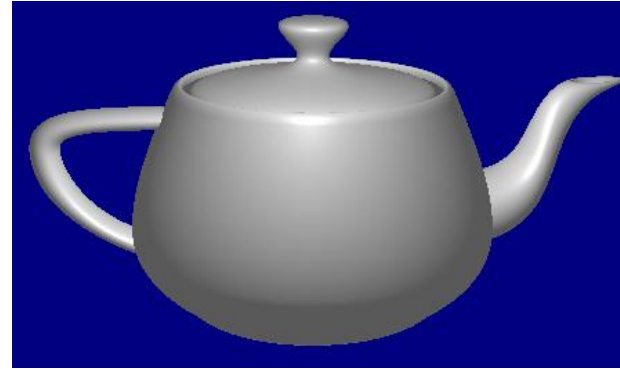


Rendering



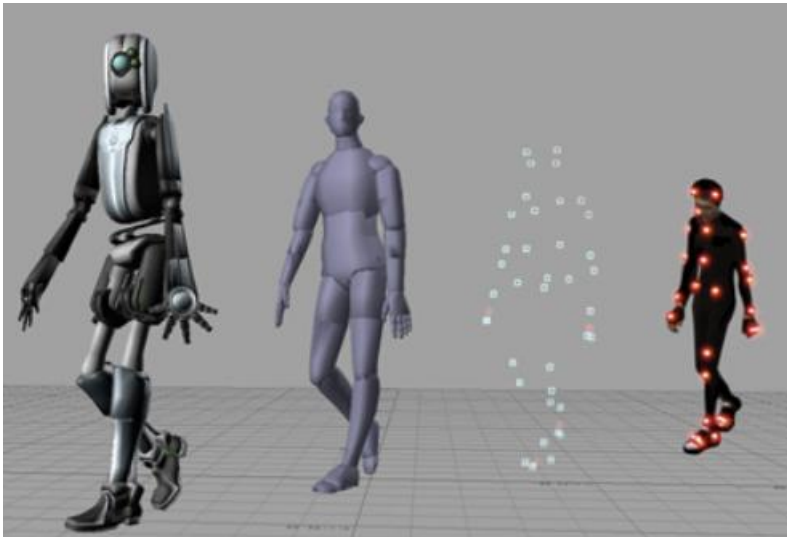
Modeling

- ▶ Spline curves, surfaces: 70^s - 80^s
- ▶ Utah teapot: Famous 3D model
- ▶ More recently: Triangle meshes often acquired from real objects



Modeling

- ▶ In 3D computer graphics, 3D modeling is the process of developing a mathematical coordinate-based representation of any surface of an object (inanimate or living) in three dimensions via specialized software by manipulating edges, vertices, and polygons in a simulated 3D space.



Simply the process of
creating 3D objects

Rendering: 1960s (visibility)

- ▶ Roberts (1963), Appel (1967) - hidden-line algorithms
- ▶ Warnock (1969), Watkins (1970) - hidden-surface
- ▶ Sutherland (1974) - visibility = sorting

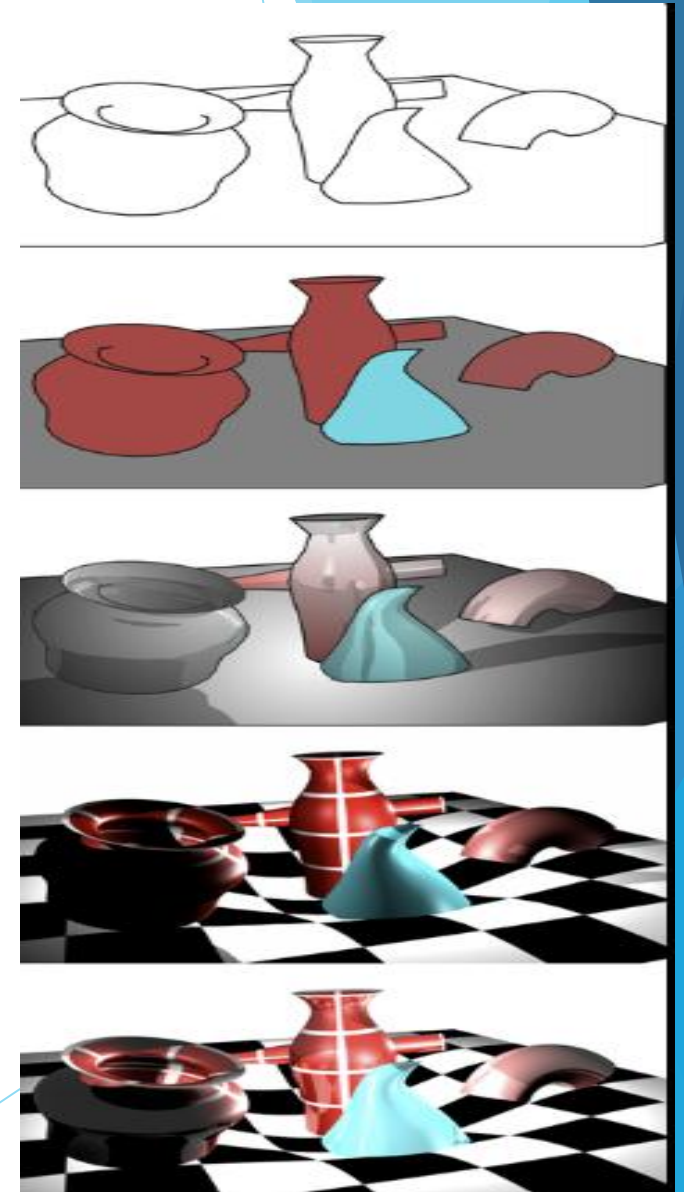


Images from FvDFH, Pixar's Shutterbug
Slide ideas for history of Rendering courtesy Marc Levoy

Rendering

- ▶ Rendering or **image synthesis** is the process of generating a **photorealistic or non-photorealistic image** from a 2D or 3D model by means of a computer program. The resulting image is referred to as the render.

Fig. A variety of rendering techniques applied to a single 3D scene



Rendering: 1970s (lighting)

1970s - raster graphics

- ▶ Gouraud (1971) - diffuse lighting, Phong (1974) - specular lighting
- ▶ Blinn (1974) - curved surfaces, texture
- ▶ Catmull (1974) - Z-buffer hidden-surface algorithm



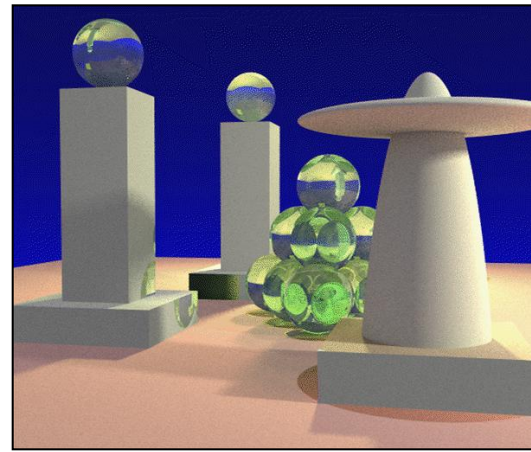
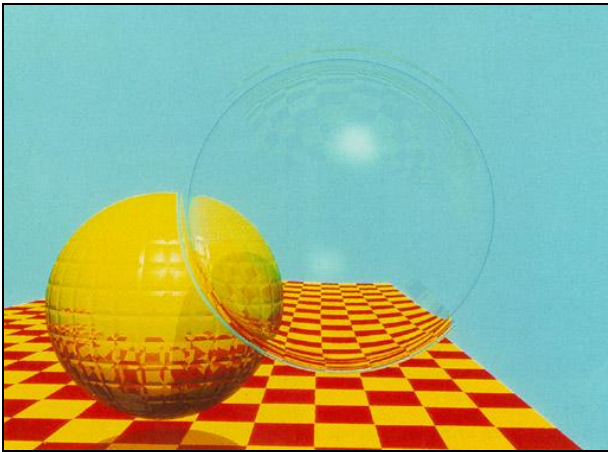
Rendering (1980s, 90s: Global Illumination)

Early 1980s - global illumination

Whitted (1980) - ray tracing

Goral, Torrance et al. (1984) radiosity

Kajiya (1986) - the rendering equation



History of Animation

- ▶ Computer animation is the process used for digitally generating animations. The more general term computer-generated imagery (CGI) encompasses both static scenes (still images) and dynamic images (moving images), while computer animation only refers to moving images. Modern computer animation usually uses 3D computer graphics to generate a three-dimensional picture. The target of the animation is sometimes the computer itself, while other times it is film.
- ▶ [History of Computer Animation - P1 - YouTube](#)

Other Areas

- ▶ **User interaction** deals with the interface between input devices such as **mouse and tablets**, the application, feedback to the user in imagery, and other sensory feedback. Historically, this area is associated with graphics largely because graphics researchers had some of the earliest access to the input/output devices that are now **ubiquitous**.
- ▶ **Virtual reality** attempts to **immerse the user into a 3D virtual world**. This typically requires at least stereo graphics and response to head motion. For true virtual reality, sound and force feedback should be provided as well. Because this area requires advanced 3D graphics and advanced display technology, it is often closely associated with graphics.
- ▶ **Visualization** attempts to give users insight into complex information via **visual display**. Often there are graphic issues to be addressed in a visualization problem.

Other Areas

- ▶ **Image processing** deals with the manipulation of 2D images and is used in both the fields of graphics and vision.
- ▶ **3D scanning** uses range-finding technology to create measured 3D models. Such models are useful for creating rich visual imagery, and **the processing of such models** often requires **graphics algorithms**.
- ▶ **Computational photography** is the use of **computer graphics, computer vision, and image processing methods** to enable new ways of photographically capturing objects, scenes, and environments.

Why Study 3D Computer Graphics?

- ▶ Fundamental Intellectual Challenges
 - ▶ Create and interact with realistic virtual world
 - ▶ Requires understanding of all aspects of physical world
 - ▶ New computing methods, displays, technologies
- ▶ Technical Challenges
 - ▶ Math of (perspective) projections, curves, surfaces
 - ▶ Physics of lighting and shading
 - ▶ 3D graphics software programming and hardware

Major Applications

- ▶ The major consumers of computer graphics technology include the following industries:
- ▶ **Video games** increasingly use sophisticated 3D models and rendering algorithms.
- ▶ **Cartoons** are often rendered directly from 3D models. Many traditional 2D cartoons use backgrounds rendered from 3D models, **which allow a continuously moving viewpoint without huge amounts of artist time.**
- ▶ **Visual effects** use almost all types of computer graphics technology. Almost **every modern film uses digital compositing to superimpose backgrounds with separately filmed foregrounds.** Many films also use 3D modeling and animation to create synthetic environments, objects, and even characters that most viewers will never suspect are not real.

Major Applications

- ▶ **Animated films** use many of the same techniques that are used for visual effects, but without necessarily aiming for images that look real.
- ▶ **CAD/CAM** stands *design* and *manufacturing*. These fields use computer technology to design parts and products on the computer and then, using these virtual designs, to guide the manufacturing process. For example, many mechanical parts are designed in a 3D computer modeling package and then automatically produced on a computer-controlled milling device.
- ▶ **Simulation** can be thought of as accurate video gaming. For example, a flight simulator uses sophisticated 3D graphics to simulate the experience of flying an airplane. Such simulations can be extremely useful for initial training in safety-critical domains such as driving, and for scenario training for experienced users such as specific fire-fighting situations that are too costly or dangerous to create physically

Major Applications

- ▶ **Medical imaging** creates meaningful images of scanned patient data. For example, a computed tomography (CT) dataset is composed of a large 3D rectangular array of density values. **Computer graphics** is used to create shaded images that help doctors extract the most salient information from such data.
- ▶ **Information visualization** creates images of data that do not necessarily have a “natural” visual depiction. For example, **the temporal trend of the price of ten different stocks** does not have an obvious visual depiction, but clever graphing techniques can help humans see the patterns in such data

Graphics APIs

- ▶ A key part of using graphics libraries is dealing with a *graphics API*. An *application program interface* (API) is a **standard collection of functions to perform** a set of related operations, and a graphics API is a set of functions that perform basic operations such as drawing images and 3D surfaces into windows on the screen.
- ▶ Every graphics program needs to be able to use two related APIs: **a graphics API for visual output and a user-interface API to get input from the user**. There are currently two dominant paradigms for graphics and user-interface APIs. The first is the integrated approach, exemplified by Java, where the graphics and user interface toolkits are integrated and portable *packages* that are fully standardized and supported as part of the language.
- ▶ The second is represented by **Direct3D and OpenGL**, where the drawing commands are part of a software library tied to a language such as C++, and the user-interface software is an independent entity that might vary from system to system

Graphics Pipeline

- ▶ Every desktop computer today has a powerful 3D *graphics pipeline*. This is a special software/hardware subsystem that efficiently draws 3D primitives in perspective. Usually these systems are optimized for processing 3D triangles with shared vertices. The basic operations in the pipeline map the 3D vertex locations to 2D screen positions and shade the triangles so that they both look realistic and appear in proper back-to-front order.

Graphics Pipeline

- ▶ Although drawing the triangles in valid back-to-front order was once the most important research issue in computer graphics, it is now almost always solved using the *z-buffer*, which uses a special memory buffer to solve the problem in a brute-force manner.
- ▶ It turns out that the geometric manipulation used in the graphics pipeline can be accomplished almost entirely in a 4D coordinate space composed of three traditional geometric coordinates and a fourth *homogeneous* coordinate that helps with perspective viewing.

Numerical Issues

- ▶ Many graphics programs are really just 3D numerical codes. Numerical issues are often crucial in such programs. In the “old days,” it was very difficult to handle such issues in a robust and portable manner because machines had different internal representations for numbers, and even worse, handled exceptions in different and incompatible ways. Fortunately, almost all modern computers conform to the *IEEE floating-point* standard (IEEE Standards Association, 1985).
- ▶ 1. **Infinity (∞)**. This is a valid number that is larger than all other valid numbers.
- ▶ 2. **Minus infinity ($-\infty$)**. This is a valid number that is smaller than all other valid numbers.
- ▶ 3. **Not a number (NaN)**. This is an invalid number that arises from an operation with undefined consequences, such as zero divided by zero.

Numerical Issues

- ▶ The designers of IEEE floating-point made some decisions that are extremely convenient for programmers. Many of these relate to the three special values above in handling exceptions such as division by zero. In these cases an exception is logged, but in many cases the programmer can ignore that. Specifically, for any positive real number a , the following rules involving division by infinite values

$$+a/(+\infty) = +0,$$

$$-a/(+\infty) = -0,$$

$$+a/(-\infty) = -0,$$

$$-a/(-\infty) = +0.$$

Efficiency

- ▶ Efficiency is achieved through careful tradeoffs, and these tradeoffs are different for different architectures. However, for the foreseeable future, a good heuristic is that programmers should pay more attention to memory access patterns than to operation counts .
- ▶ A reasonable approach to making code fast is to proceed in the following order, taking only those steps which are needed:

Efficiency

- ▶ 1. Write the code in the most straightforward way possible. Compute intermediate results as needed on the fly rather than storing them.
- ▶ 2. Compile in optimized mode.
- ▶ 3. Use whatever profiling tools exist to find critical bottlenecks.
- ▶ 4. Examine data structures to look for ways to improve locality. If possible, make data unit sizes match the cache/page size on the target architecture.
- ▶ 5. If profiling reveals bottlenecks in numeric computations, examine the assembly code generated by the compiler for missed efficiencies. Rewrite source code to solve any problems you find.

Class Design

- ▶ A key part of any graphics program is to have good classes or routines for geometric entities such as vectors and matrices, as well as graphics entities such as RGB colors and images
This implies that some basic classes to be written include:
- ▶ **vector2**. A 2D vector class that stores an x - and y -component. It should store these components in a length-2 array so that an indexing operator can be well supported. You should also include operations for vector addition, vector subtraction, dot product, cross product, scalar multiplication, and scalar division.
- ▶ **vector3**. A 3D vector class analogous to **vector2**.
- ▶ **hvector**. A homogeneous vector with four components.
- ▶ **rgb**. An RGB color that stores three components. You should also include operations for RGB addition, RGB subtraction, RGB multiplication, scalar multiplication, and scalar division.

Class Design

- ▶ **transform**. A 4×4 matrix for transformations. You should include a matrix multiply and member functions to apply to locations, directions, and surface normal vectors.
- ▶ **image**. A 2D array of RGB pixels with an output operation.
- ▶ In addition, you might or might not want to add classes for intervals, orthonormal bases, and coordinate frames.

Float vs. Double

- ▶ Modern architecture suggests that keeping memory use down **and maintaining coherent memory access are the keys to efficiency**. This suggests using single precision data. However, avoiding numerical problems suggests using double precision arithmetic. The tradeoffs depend on the program, but it is nice to have a default in your class definitions.

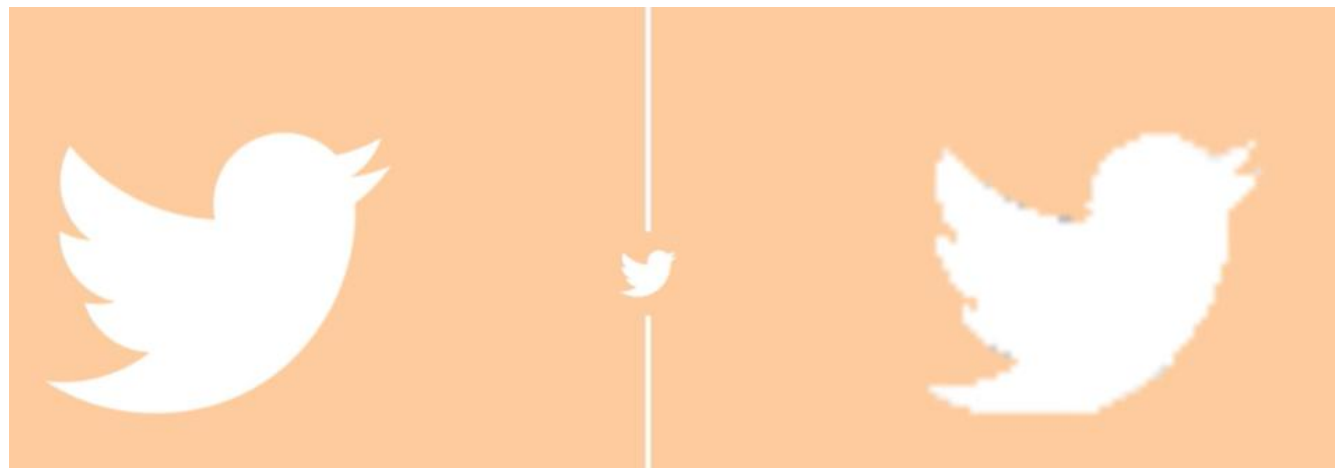
Raster Images

- ▶ Most computer graphics images are presented to the user on some kind of *raster display*. Raster displays show images as rectangular arrays of *pixels*. A common *Pixel* is short for “picture element.”
- ▶ example is a flat-panel computer display or television, which has a rectangular array of small light-emitting pixels that can individually be set to different colors to create any desired image
- ▶ Because rasters are so prevalent in devices, *raster images* are the most common way to store and process images. A raster image is simply a 2D array that stores the *pixel value* for each pixel—usually a color stored as three numbers, for red, green, and blue. A raster image stored in memory can be displayed by using each pixel in the stored image to control the color of one pixel of the display.

vector image

- ▶ A *vector image* is described by storing descriptions of shapes—areas of color bounded by lines or curves—with no reference to any particular pixel grid. In essence this amounts to storing the *instructions* for displaying the image rather than the pixels needed to display it. The main advantage of vector images is that they are *resolution independent* and can be displayed well on very high resolution devices.

Vector image vs Raster Images



Raster Devices

- Output
 - Display
 - * Transmissive: liquid crystal display (LCD)
 - * Emissive: light-emitting diode (LED) display
 - Hardcopy
 - * Binary: ink-jet printer
 - * Continuous tone: dye sublimation printer
- Input
 - 2D array sensor: digital camera
 - 1D array sensor: flatbed scanner

Displays

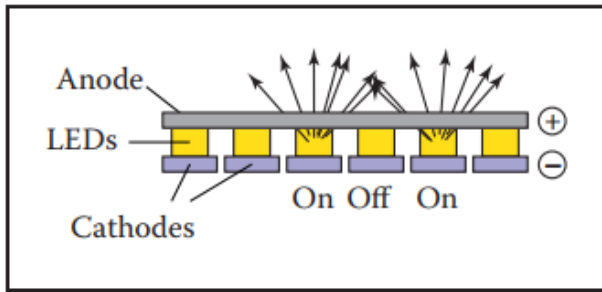


Figure 3.1. The operation of a light-emitting diode (LED) display.

The pixels in a color display are divided into three independently controlled *subpixels*—one red, one green, and one blue—each with its own LED made using different materials so that they emit light of different colors

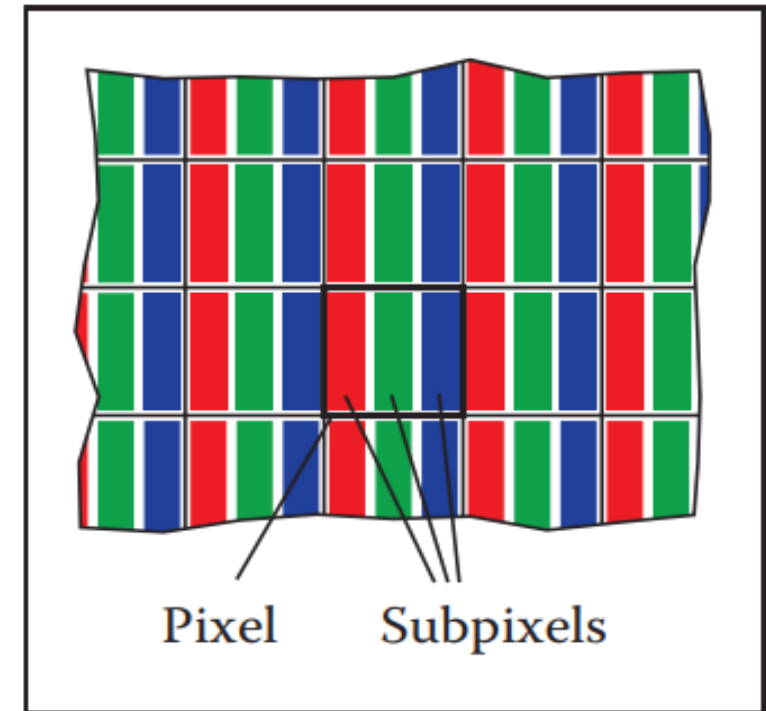


Figure 3.2. The red, green, and blue subpixels within a pixel of a flat-panel display.

Displays

- ▶ Current displays, including **televisions and digital cinematic projectors** as well as displays and projectors for computers, are nearly universally **based on fixed arrays of pixels**. They can be separated into emissive displays, which use pixels that directly emit controllable amounts of light, and transmissive displays, in which the pixels themselves don't emit light but instead vary the amount of light that they allow to pass through them.
- ▶ **Light-emitting diode (LED) displays** are an example of the emissive type. Each pixel is composed of one or more **LEDs, which are semiconductor devices** (based on inorganic or organic semiconductors) that emit light with intensity **depending on the electrical current passing through them**

Displays

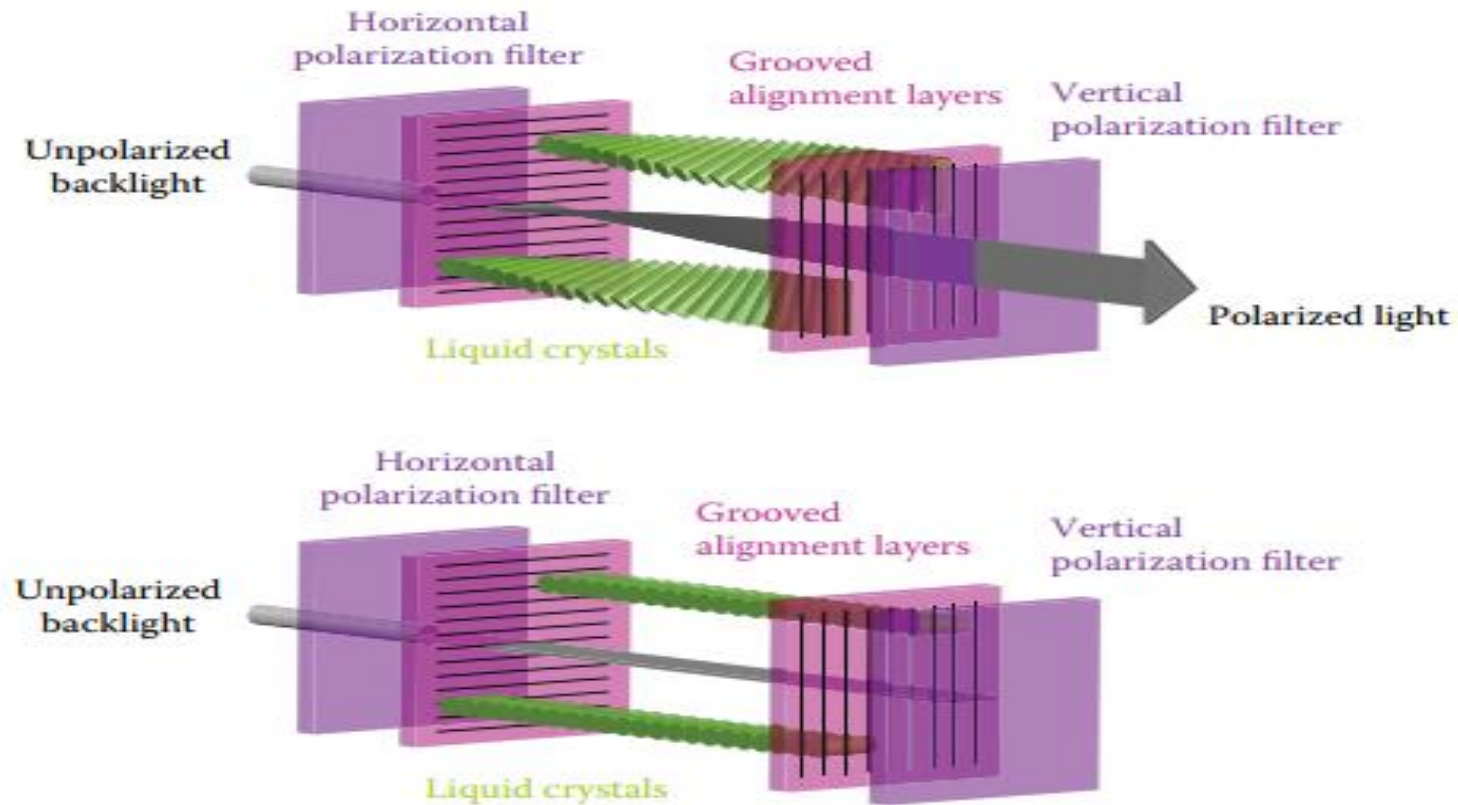


Figure 3.3. One pixel of an LCD display in the off state (bottom), in which the front polarizer blocks all the light that passes the back polarizer, and the on state (top), in which the liquid crystal cell rotates the polarization of the light so that it can pass through the front polarizer. *Figure courtesy Erik Reinhard (Reinhard, Khan, Akyüz, & Johnson, 2008)*

Displays

- ▶ Liquid crystal displays (LCDs) are an example of the transmissive type. A liquid crystal is a material whose molecular structure enables it to rotate the polarization of light that passes through it, and the degree of rotation can be adjusted by an applied voltage. An LCD pixel (Figure 3.3) has a layer of polarizing film behind it, so that it is illuminated by polarized light—let's assume it is polarized horizontally

Rasterization

- ▶ Rasterization is the task of taking an image described in a vector graphics format (shapes) and converting it into a raster image (pixels or dots) for output on a video display or printer, or for storage in a bitmap file format. It refers to both rasterization of models and 2D rendering primitives such as polygons, line segments, etc.

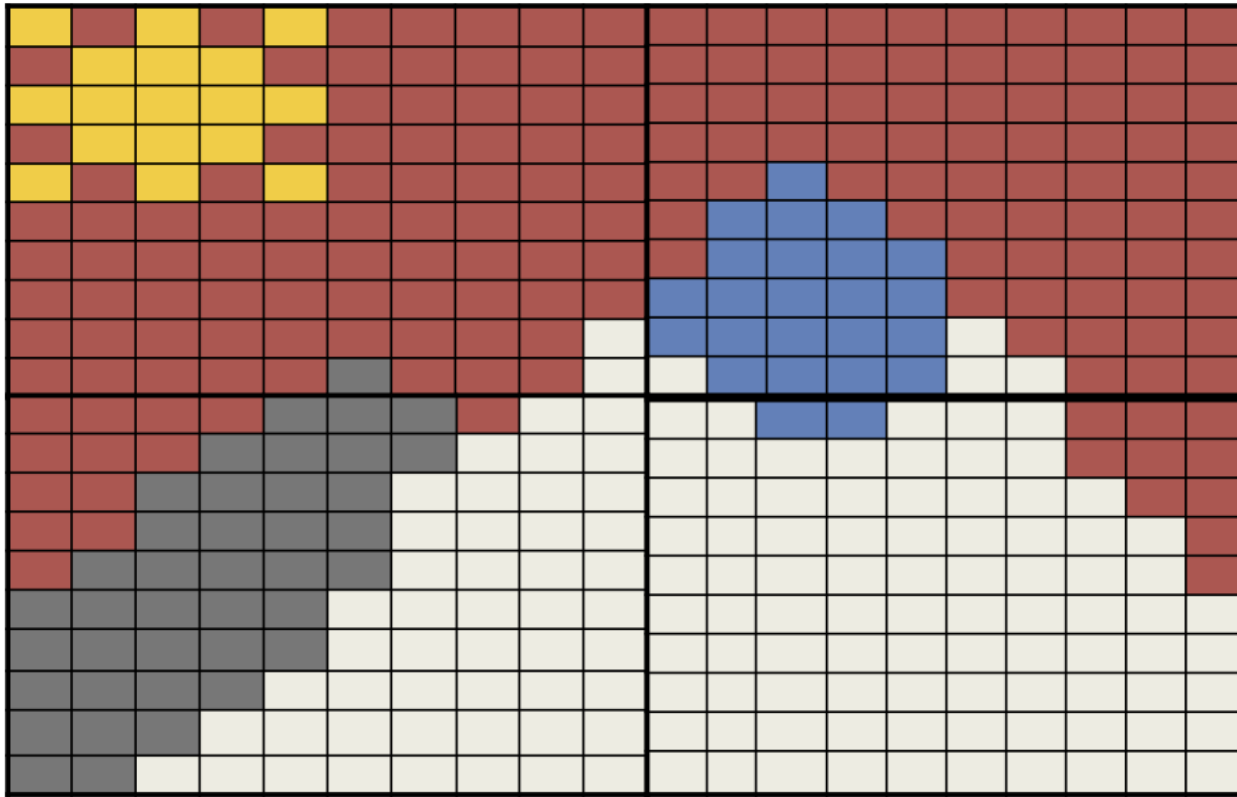
Bitmapped Graphics

- ▶ There are two basic types of graphics:
 - **Bitmapped** and **Vector**
- ▶ Bitmapped graphics are much more common
- ▶ Often they are called raster graphics
- ▶ When you create a bitmapped graphic you are basically creating a bunch of colored dots

Bitmapped Graphics

- ▶ The bitmapped graphic is stored as an array of dots, or pixels
- ▶ Each pixel gets assigned a specific color
- ▶ The more pixels you have, the more detailed the image can be
 - Imagine only have one pixel, all you get is a dot
- ▶ Some common bitmap graphics programs are:
 - Photoshop
 - Paint Shop Pro
 - GIMP
 - Photo-Paint
 - Graphic Converter

Exaggerated Example of a Bitmap Image



Alpha Compositing

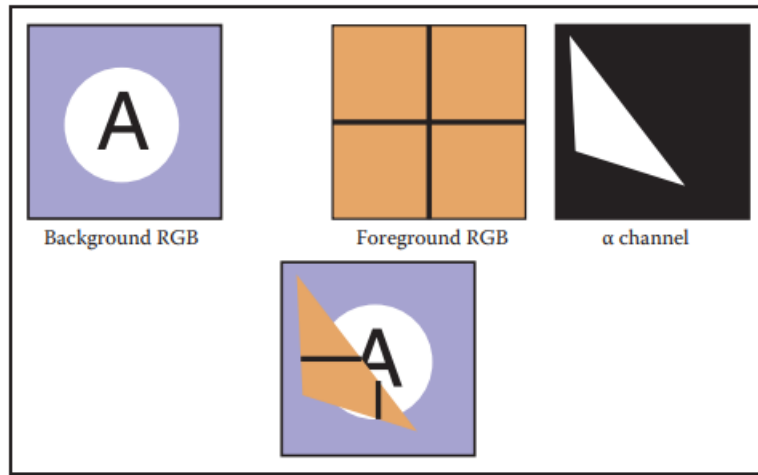
- ▶ Often we would like to only partially overwrite the contents of a pixel. A common example of this occurs in *compositing*, where we have a background and want to insert a foreground image over it. For opaque pixels in the foreground, we just replace the background pixel. For entirely transparent foreground pixels, we do not change the background pixel. For *partially* transparent pixels, some care must be taken. Partially transparent pixels can occur when the foreground object has partially transparent regions, such as glass.
- ▶ The most important piece of information needed to blend a foreground object over a background object is the *pixel coverage*, which tells the fraction of the pixel covered by the foreground layer. We can call this fraction α .



Alpha Compositing

- ▶ to composite a foreground color $\mathbf{c}_f$ over background color $\mathbf{c}_b$, and the fraction of the pixel covered by the foreground is α , then we can use the formula

$$\mathbf{c} = \alpha \mathbf{c}_f + (1 - \alpha) \mathbf{c}_b. \quad (3.2)$$



Since the weights of the foreground and background layers add up to 1, the color won't change if the foreground and background layers have the same color.

Figure 3.14. An example of compositing using Equation (3.2). The foreground image is in effect cropped by the α channel before being put on top of the background image. The resulting composite is shown on the bottom.

Image Storage

- ▶ • **jpeg**. This lossy format compresses image blocks based on thresholds in the human visual system. This format works well for natural images.
- ▶ • **tiff**. This format is most commonly used to hold binary images or losslessly compressed 8- or 16-bit RGB although many other options exist.
- ▶ • **ppm**. This very simple lossless, uncompressed format is most often used for 8-bit RGB images although many options exist.
- ▶ • **png**. This is a set of lossless formats with a good set of open source management tools

Practice code

```
pip install svgwrite (if necessary)
```

```
import svgwrite

# Create an SVG drawing object
svg = svgwrite.Drawing("vector_image.svg", size=("400px", "400px"))

# Add a rectangle (x, y, width, height)
svg.add(svg.rect(insert=(50, 50), size=("200px", "100px"),
fill="blue", stroke="black", stroke_width=3))

# Add a circle (center_x, center_y, radius)
svg.add(svg.circle(center=(300, 150), r=50, fill="red",
stroke="black", stroke_width=2))

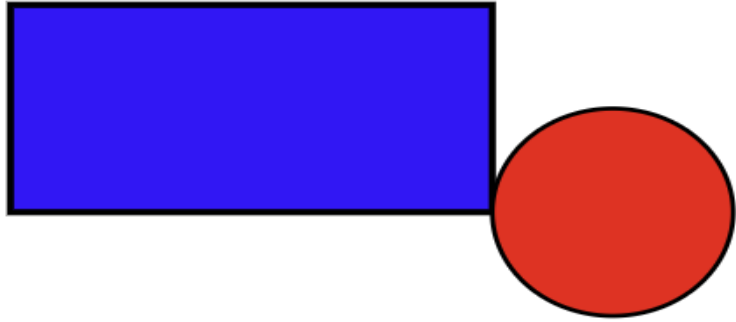
# Add a line (start_x, start_y, end_x, end_y)
svg.add(svg.line(start=(50, 300), end=(350, 300), stroke="green",
stroke_width=4))

# Add some text
svg.add(svg.text("Vector Image Example", insert=(100, 380),
fill="black", font_size="20px"))

# Save the drawing to a file
svg.save()

print("Vector image saved as vector_image.svg")
```

Output



Vector Image Example

Practice code

```
from PIL import Image, ImageDraw

# Create a blank white image (width, height) with RGB mode
width, height = 400, 400

image = Image.new("RGB", (width, height), "white")

# Create a draw object
draw = ImageDraw.Draw(image)

# Draw a rectangle (top left corner, bottom right corner)
draw.rectangle([(50, 50), (200, 150)], outline="black", fill="blue", width=3)

# Draw an ellipse (bounding box for the ellipse)
draw.ellipse([(250, 100), (350, 200)], outline="black", fill="red", width=3)

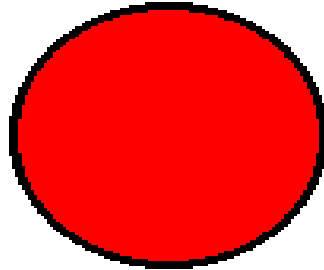
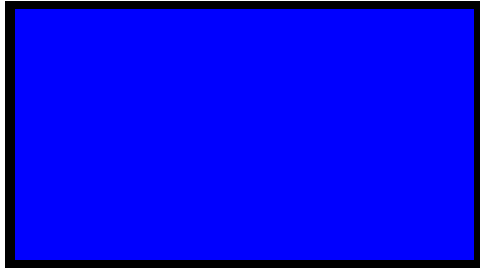
# Draw a line (start point, end point)
draw.line([(50, 300), (350, 300)], fill="green", width=4)

# Draw text on the image
draw.text((100, 350), "Raster Image Example", fill="black")

# Save the image as a PNG file
image.save("raster_image.png")

print("Raster image saved as raster_image.png")
```

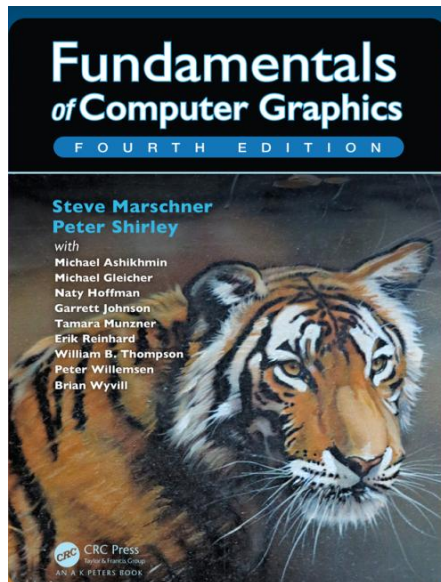
Output



Raster Image Example

Reference

- ▶ Chapter 1 and chapter 3



[History of Computer Animation - P1 - YouTube](#)

Question



Thank you

